Project 3: SSD Performance Profiling

Samson Kempiak
ECSE 4320/6320 Advanced Computer Systems
kempis@rpi.edu

October 16, 2026

Abstract

We profile a Micron 2210 512 GB QLC NVMe SSD on a PCIe 3.0 ×4 link with `O_DIRECT` I/O against disposable files, using a `libaio` benchmark as the primary engine and FIO as a cross-check. At QD=1, 4 KiB random reads take 53.2 µs (18 783 IOPS) and writes 16.0 µs, because the write buffer hides the flash program. With one request in flight, sequential reads level off near 1 444 MiB/s. That limit comes from per-request latency, not the interface. At QD=32 they top out at 2 144 MiB/s, only 57 % of the 3756 MiB/s raw PCIe link ceiling, so the NAND back end sets the limit. Random-read IOPS saturate at QD 64 (149 422 IOPS) under a stated knee criterion, and Little's Law serves as a consistency check on the closed-loop harness. Writes land in an SLC cache at 1 071 MiB/s. In a lean re-run on a freshly trimmed, idle drive, that cache absorbed only 3.8 GiB of a sequential fill, far less than the ∼20 GiB we expected and consistent with a dynamic SLC cache sized by the drive's free space. Past it, sequential writes fall to 63 MiB/s and 4 KiB random writes to about 15k IOPS. On a rested drive, read tails widen with queue depth to a p99.9 of 5.7× the median at QD=128; in an earlier run that followed heavy writes they reached 24×. Sequential reads of randomly written data run 6–9× slower. The required misleading setup, dropping `O_DIRECT`, over-reports throughput 19× because it measures the page cache.

1 Introduction

Modern NVMe SSDs deliver high IOPS and throughput only when requests are issued concurrently. Like memory, storage shows a queueing-driven throughput–latency tradeoff: deeper queues raise device utilization up to a saturation knee, and past it latency climbs sharply for little extra throughput. This project characterizes that behaviour on a Micron 2210 512 GB QLC NVMe SSD. It measures QD=1 latency, block-size and read/write-mix effects, the queue-depth knee, tail latency and device state (working set, SLC-cache exhaustion, temperature). All I/O is `O_DIRECT` against disposable files. The primary engine is a small `libaio` benchmark, and FIO cross-checks it (Sections 2 and 3).

This report answers:

1. What are the QD=1 latencies for small random and large sequential I/O? (Section 4)
2. How do block size, access pattern, read/write mix and queue depth shape IOPS, MiB/s and latency, and where is the PCIe interface ceiling? (Sections 5 and 6)
3. Where is the queue-depth knee, by what criterion, and what does Little's Law say about the harness? (Section 7)

Table 1: Measurement environment.

CPU	Intel(R) Core(TM) i9-10900F CPU @ 2.80GHz (x86_64)
SIMD ISA	sse sse2 ssse3 sse4_1 sse4_2 avx avx2 fma f16c – widest 256-bit (ymm)
Topology	1 socket(s), 10 cores/socket, 1 threads/core (10 of 20 logical CPUs online); NUMA: 1 node(s)
SMT	off
Caches	L1d 320 KiB (10 instances); L2 2.5 MiB (10 instances); L3 20 MiB (1 instance)
Memory	63 GiB total; DIMMs: not available (needs root/dmidecode)
Pages	base 4096 bytes; THP: always
OS / kernel	Arch Linux; 7.2.9-arch1-1
Frequency policy	governor: performance; driver: intel_pstate; turbo: disabled; EPP: performance
Pinning / isolation	a single I/O-issuing thread, not CPU-pinned (storage I/O is device-bound, not CPU-bound, below the host-saturation queue depths of Section 7); no isolcpus
Compilers	gcc (GCC) 16.2.1 20260810; clang version 23.1.3
Tools	perf: perf version 7.2.11-1 (perf_event_paranoid 0); FIO: fio-3.42; MLC: present
GPU (optional)	NVIDIA GeForce RTX 3070; CUDA not available (no CUDA)
SSD	Micron 2210 `MTFDHBA512QFD`, 512 GB QLC NVMe, firmware `P6MA001` (from `/sys/class/nvme/nvme0`; SMART logs need root)
Interface	PCIe link 8.0 GT/s ×4 (Gen3), both current and maximum, read from `current_link_speed`/`current_link_width` in sysfs. Raw ceiling $8\,\mathrm{GT/s} \times 4 \times \frac{128}{130} = 3.94\,\mathrm{GB/s}$ (3756 MiB/s), roughly 3.2 GB/s to 3.5 GB/s after TLP/DLLP overhead
Block layer	scheduler `none`, `nr_requests` 1023, 16 I/O queues, volatile write cache reported as "write back"
Target	disposable files on the SSD's ext4 `/` partition (`nvme0n1p3`, mounted `relatime`, no online discard), reached via `ACS_SCRATCH`: a 2 GiB test file for the sweeps, and a 60 GiB file for the original device-state runs. The lean re-run used a 4 GiB test file, with random offsets over its first 2 GiB as before, and a fill file capped at 40 GiB
Temperature	NVMe composite sensor `/sys/block/nvme0n1/device/hwmon*/temp1_input`, readable without root, logged per repetition and every second in the sustained runs
Env. snapshot	2026-10-21 20:46 EDT (each raw run directory also stores its own `env.txt`)

4. What do tail latency and device state reveal: working-set size, SLC-cache exhaustion and steady state, data layout, and temperature? (Sections 8 and 9)

5. Which setup mistake produces misleading numbers? (Section 10)

In summary, QD=1 random reads cost 53.2 µs and writes only 16.0 µs, because the write buffer absorbs them. With one request in flight, sequential reads are limited by per-request latency rather than the PCIe 3.0 ×4 link. Deeper queues raise them to only 57 % of the link, where the NAND back end becomes the limit (Section 5). The random-read knee sits at QD 64. Tails stretch with queueing and with background flash work. Once the SLC cache is full, sustained writes drop 17× to the direct-to-QLC rate, and the drive's recent write history changes both read tails and sequential-read speed (Section 9). Buffered I/O over-reports throughput by more than an order of magnitude because it measures the page cache, not the device.

2 Methodology and Safety

2.1 Platform

The machine has two drives: this Micron NVMe SSD and a 2 TB SATA HDD (`/dev/sda`, mounted at `/data`). The scratch directory is placed on the SSD's filesystem. Each run's `env.txt` resolves

the scratch filesystem's partition to its parent disk (`nvme0n1p3`→`nvme0n1`) and records that disk's `rotational` flag (0), model, firmware, PCIe link and temperature, so a run can never be silently mislabelled (Section 12 describes how this check once failed).

2.2 Data-loss safeguards

- **Targets.** Only disposable files (`testfile.bin`, `bigfile.bin`) in a dedicated scratch directory are ever written; nothing else on the SSD is touched. Both files are deleted when `run.sh` exits, including on interrupt.
- **Gate.** `scripts/safety.sh` runs before every experiment. It hard-refuses a block or character device (including through a symlink), a mountpoint, a directory, any path outside the scratch directory, and a target without enough free space. `scripts/validate.sh` (`make test`) verifies that it refuses `/dev/nvme0n1` and `/dev/sda`, a symlink to `/dev/sda`, `/etc/passwd` and `/`. These checks only `stat` the paths and never open them.
- **Benchmark.** `blkbench` `lstat`s its target *before* opening it, refuses anything that is not a regular file, and opens with `O_NOFOLLOW`. No raw device or partition is ever opened.
- **Space.** The large device-state file (60 GiB in the original run, 40 GiB in the lean re-run) is refused unless the filesystem keeps at least 10 GiB free after creating it.

2.3 I/O engines and configuration

The primary engine is `build/blkbench`, a small O_DIRECT benchmark built on Linux `libaio`, the same asynchronous engine as FIO's `libaio` ioengine [1]. It keeps exactly QD requests in flight in a closed loop: every completion is immediately replaced. It records the submit-to-completion latency of every I/O. FIO 3.42 runs the same points through `fio/job.fio` (`libaio`, `direct=1`, total-latency percentiles), and Section 3 compares the two engines. `ENGINE=fio` routes every standard sweep through FIO instead.

Engine	`libaio`; `O_DIRECT` (page cache bypassed); 4 KiB-aligned buffers and block-aligned offsets; 64-bit uniform random offsets (splitmix64)
Timing	4 s per repetition (6 s for tail runs). Points that write also stop after 256 MiB per repetition, which bounds the bytes written (Table 2); 128 MiB in the lean re-run, which also waits 15 s before each write point. No separate ramp: preconditioning absorbs warm-up
Repetitions	5 per point (3 for the block-size write points, also to bound bytes written; in the lean re-run 4 per engine for the FIO write points and 4 per working-set span); figures and tables show the median with inter-quartile-range (IQR) bars
Data pattern	incompressible: every write buffer is filled once, before timing, from an xorshift64* generator seeded per queue slot; the preconditioning pass uses the same generator. `make test` checks that `gzip` cannot compress the result
Latency metric	per-I/O submit→completion latency; p50/p95/p99/p99.9 from the full sorted sample, with no reservoir truncation
Preconditioning	one sequential O_DIRECT write pass over the `fallocate`d test file, repeated before every *sequential-read* group, because random writes scatter the physical layout (Section 9). The current `run.sh` also preconditions before *every* experiment, including inside `all`. In the reported 2026-10-21 run, `all` preconditioned only once, before `zero_queue`. Later experiments started from the file the previous one left; only the sequential-read groups of `zero_queue`, `block_size`, `pitfall` and `layout` were re-filled. `rerun_clean.sh` (Section A) preconditions before each experiment. Its lean mode preconditions the 4 GiB test file once, before `tail`; the later lean steps (random reads and writes only) run on that file
Temperature	NVMe composite temperature recorded with every repetition (`temp_c`)
Driver	`SSD_Performance_Profiling/scripts/run.sh`; benchmark `src/blkbench.c`

Because the issuing thread only submits and reaps completions, one thread can keep the device's queue full. The Little's-Law check in Section 7 shows where this stops being true.

2.4 Execution and run order

The timed runs were executed by the author in a terminal, using the scripts described here, with the CPU governor at `performance`, turbo off and SMT off. `run.sh all` ran the experiments in the order of Table 2 on 2026-10-21, from 10:32 to 10:54. `SUSTAIN_GIB=60 run.sh device_state` followed immediately, from 10:54 to 11:23. Harness development and a discarded pilot run took place from 2026-10-12 to 2026-10-20; all reported numbers come from the 2026-10-21 runs. In those two runs the drive's state carried over from one experiment to the next, and nothing in between trimmed or idled it: the SLC-cache fill level and the background folding/GC backlog were both inherited. Several results depended on this: the write collapses in Sections 5 and 7, the tail runs in Section 8, and the whole of Section 9.

Lean re-run. Those experiments were therefore measured again the same evening with `rerun_clean.sh lean` (run `20261021-220614`; Table 3). The author ran `sudo fstrim -v /` at 21:34:54 and left the machine idle for at least 30 min. The script's own idle check then passed (over 10 s: 0 MiB read, 1 MiB written, 0 % busy), with 77 GiB free on the scratch filesystem. The run started at 22:06 and ended at 22:32:17 EDT, with the governor at `performance`, turbo off, SMT off and the ACPI GPE 0x42 interrupt source masked. The sequence puts the sustained fill first, on the rested drive, and stops it soon after the cliff instead of writing 60 GiB. After that, it rests 10 min before the tail runs and 5 min before the large random writes. Every write point is capped at 128 MiB per repetition and preceded by a 15 s pause. Each lean measurement replaces only the matching configurations of the earlier run, and every other value in the report still comes from

Table 2: Measured wall-clock time of each experiment in the original 2026-10-21 runs, in execution order, and the maximum bytes it can write with the default knobs (byte caps plus preconditioning).

Experiment	Time	Max written	Notes
`zero_queue`	1.1 min	7 GiB	includes the initial preconditioning
`block_size`	9.1 min	22 GiB	48 points; write points 3 reps, 256 MiB cap
`rw_mix`	0.5 min	4 GiB	
`queue_depth`	3.7 min	11 GiB	read and write, QD 1–256
`tail`	2.0 min	–	reads only
`pitfall`	0.8 min	2 GiB	reads only (plus re-fill)
`layout`	2.2 min	6 GiB	includes a 4 GiB random overwrite
`fio_check`	3.3 min	8 GiB	both engines, interleaved
`sustained`	24.9 min	72 GiB	60 GiB fill + ≤8 GiB random + ≤4 GiB recovery
`working_set`	4.0 min	5 GiB	on the 60 GiB file left by `sustained`

Table 3: The lean re-run (`rerun_clean.sh lean`, 2026-10-21, 22:06–22:32 EDT) in execution order: wall-clock time from the run log, including rests, and the most each step could write.

Step	Time	Max written	What it re-measured
`sustained`	1.6 min	6.3 GiB	fill (stopped at 4.3 GiB), 1 GiB random, 60 s idle, 1 GiB recovery
`tail`	12.1 min	4 GiB	10 min rest, fresh 4 GiB precondition, QD 8/32/64/128 reads
`working_set`	1.6 min	0.75 GiB	spans 1, 2, 4 GiB, forward then reverse
`block_size`	8.5 min	1.1 GiB	QD=32 random reads (8 sizes), 5 min rest, QD=1 random writes ≥256 KiB
`queue_depth`	1.1 min	2.5 GiB	random writes at QD 4–32
`fio_check`	1.0 min	3 GiB	the three write points, engine order alternating

`20261021-103201`. Figure and table captions name the run their points come from.

3 Validation and Sanity Checks

Before trusting any number, we confirm that the benchmark describes the SSD and not an artifact. `make -C SSD_Performance_Profiling test` (`scripts/validate.sh`) runs the automated checks; it writes about 70 MiB and deletes it.

- **Safety gate.** `safety.sh` refuses both block devices on the host (`/dev/nvme0n1`, the SSD; `/dev/sda`, the HDD), a symlink to `/dev/sda` placed inside the scratch directory, a directory, a path outside the scratch directory (`/etc/passwd`) and a mountpoint (`/`). It accepts only a disposable file inside the scratch directory. `blkbench` independently refuses a directory and a symlink before opening them. None of these checks opens a device.
- **Incompressible data.** The preconditioned file does not shrink under `gzip -1` (output ≥ 99% of input), so the drive cannot gain from compression or zero detection.
- **Physical orderings.** On a preconditioned file, 128 KiB sequential throughput exceeds 4 KiB random throughput. Raising QD from 1 to 8 raises the measured concurrency $L = \lambda W$, and L never exceeds the configured QD (a hard invariant of a closed loop with QD slots; Section 7).
- **Direct I/O is real.** With `O_DIRECT` the QD=1 4 KiB reads take tens of microseconds. The `-buffered` path reaches about two million IOPS at sub-microsecond latency because the page

Table 4: `blkbench` vs. FIO on identical points (medians of interleaved repetitions: 5 per engine for reads, 4 per engine for writes, 2 of them in each position; latencies in µs). Data: the write points from the lean re-run of 2026-10-21; all other points from the 2026-10-21 run.

Pattern	Block	QD	IOPS		FIO/blk	p50		p99	
			blk	FIO		blk	FIO	blk	FIO
rand read	4 KiB	1	18 763	18 438	0.983	49.9	50.4	58.7	57.6
rand read	4 KiB	32	129 580	129 705	1.001	217.1	216.1	663.5	692.2
seq read	128 KiB	1	1 793	1 804	1.006	546.4	544.8	1 057.9	1 036.3
seq read	128 KiB	32	1 827	1 827	1.000	17 401.3	17 432.6	19 449.4	19 791.9
rand write	4 KiB	1	66 643	59 109	0.887	14.7	15.3	16.9	17.7
rand write	4 KiB	32	226 524	254 031	1.121	121.7	119.8	150.3	154.6
seq write	128 KiB	1	8 388	8 303	0.990	99.4	98.3	259.9	286.7

cache serves it from RAM (Section 10). A gap of two orders of magnitude shows that the real runs bypass the cache. `make test` asserts that buffered re-reads are more than 5× faster than direct ones.

- **Interface ceiling known, not assumed.** sysfs reports the link at 8 GT/s ×4, so no measurement may exceed 3756 MiB/s (Section 2). The highest sequential read we measure, 2 144 MiB/s (64 KiB, QD=32) (Section 5), is checked against this bound.
- **Device identity.** Every run's `env.txt` records the parent disk of the scratch filesystem, its `rotational` flag (0), the model string `Micron_2210_MTFDHBA512QFD`, the firmware and the PCIe link.

`iostat` and `perf` are not available here, so the effective queue depth is checked in-band instead (Table 8).

3.1 Cross-check against FIO

`run.sh fio_check` runs seven points (QD=1 and QD=32; reads and writes; 4 KiB random and 128 KiB sequential) through both engines on the same preconditioned file. The engines alternate repetition by repetition, so drift affects both equally. FIO uses `fio/job.fio`: `libaio`, `direct=1`, one request per submit, total-latency (`lat`) percentiles, which is the same submit→completion quantity `blkbench` records. Table 4 compares the medians. We treat agreement within 10 % in IOPS and p50 as validating `blkbench`. FIO's percentiles come from a log-bucketed histogram (at most ∼3 % bin error), so small p99 differences are expected.

Reads agree. On all four read points, FIO/`blkbench` IOPS ratios are 0.983–1.006 and p50 latencies differ by at most 1 %, so `blkbench` is validated for reads. The two 128 KiB sequential read points ran at only ∼225 MiB/s in *both* engines. In `run.sh all`, `fio_check` directly followed `layout`, which leaves the file randomly overwritten, and the script did not rewrite it in between (since fixed). These points therefore measured the fragmented layout of Section 9.3. That does not affect the engine comparison, but their absolute values must not be read as sequential-read performance.

Writes vary more, and mostly with drive state. The write points come from the lean re-run, where the engine order alternates within each pair of repetitions. The QD=1 sequential write agrees (ratio 0.990). The two random-write points fall just outside the 10 % criterion, in opposite directions: 0.887 at QD=1 and 1.121 at QD=32. Their p50 latencies differ by less than

Table 5: Write points of `fio_check` split by engine and by position within each pair of repetitions: median IOPS of 2 repetitions per cell (data: lean re-run of 2026-10-21).

Point	blkbench 1st	blkbench 2nd	FIO 1st	FIO 2nd
4 KiB rand, QD=1	65 641	66 758	57 845	61 637
4 KiB rand, QD=32	226 524	232 289	239 456	254 031
128 KiB seq, QD=1	9 146	7 723	8 755	7 938

1 µs (14.7 vs. 15.3 and 121.7 vs. 119.8 µs). Table 5 splits each write point by engine and by position in the pair. For the same engine, the position alone moves the result by a similar amount to the engine gap. The clearest case is the sequential write: `blkbench` gave 9 146 IOPS when it ran first and 7 723 when it ran second, a larger spread than between the engines in either position. For the random writes the position effect is smaller (2–7 %) and the engine gap (6–12 %) persists in both positions. Across the two runs, however, the QD=32 random-write ratio flipped, from 0.78 (FIO slower, and always second) in the original run to 1.12 now, and the sequential ratio went from 1.05 to 0.99. A fixed engine bias would not change sign like that. We therefore attribute the write differences mainly to drive-state variance, that is, to how much buffered data the drive still had to flush when a repetition started, rather than to an engine bias.

One pattern is consistent with a small engine effect, though. At QD=1 random write, FIO is slower in *both* positions, and it was also slower in the original run (0.93), with p50 about 0.6 µs higher each time. That would fit a slightly larger per-I/O host cost in FIO, which matters only when one request is in flight. With two repetitions per cell we cannot separate it from noise. We therefore give write numbers from either engine an uncertainty of roughly 15 %, the size of the same-engine order effect. The reads, which leave no buffer to flush, agree within 2 %.

4 Zero-Queue Baselines

Hypothesis (from first principles, then compared with the data)

At QD=1 a single I/O is in flight, so latency is the full host-stack + controller + flash path. We expect 4 KiB random reads in the tens of microseconds (flash read dominates) and 128 KiB sequential transfers to be bandwidth-limited at a few hundred µs. Writes should be *faster* than reads at QD=1 because the SSD's DRAM/SLC write buffer acknowledges them before the flash program completes.

Fixed	iodepth 1; one job; `O_DIRECT`; 5 repetitions of 4 s (write points also stop at 256 MiB)
Cases	4 KiB random read, 4 KiB random write, 128 KiB sequential read, 128 KiB sequential write
Order	random read → random write → sequential re-fill of the file → sequential read → sequential write (the re-fill keeps the sequential read independent of the random writes; Section 9.3)
Script	`SSD_Performance_Profiling/scripts/run.sh zero_queue`

4.1 Analysis

The QD=1 numbers identify an NVMe SSD, not a rotational disk. 4 KiB random read latency is 53.2 µs, not the milliseconds an HDD would show, and it is dominated by the NAND read plus the NVMe/`libaio` host path, with no seek. Random *writes* take only 16.0 µs (62 217 IOPS), because the controller acknowledges them from its write buffer (SLC cache plus controller SRAM) before the flash program finishes. A QD=1 read has no such buffer to hide behind.

Table 6: QD=1 performance on the Micron 2210 NVMe SSD: medians over repetitions, with the IQR in brackets for IOPS and p99; latencies in µs. Random reads pay the full flash-read latency; random writes are faster because the write buffer absorbs them.

Pattern	Block	IOPS [IQR]	MiB/s	Mean	p50	p95	p99 [IQR]	p99.9
rand read	4 KiB	18 783 [18 767, 18 796]	73.4	53.2	49.9	50.9	62 [62, 71]	534
rand write	4 KiB	62 217 [62 067, 62 239]	243.0	16.0	14.6	15.9	18 [18, 18]	490
seq read	128 KiB	11 530 [11 511, 11 543]	1 441.2	86.7	80.5	114.2	491 [490, 496]	563
seq write	128 KiB	5 100 [891, 5 298]	637.5	196.0	90.0	1 156.4	1 437 [1 436, 34 765]	9 853

The 128 KiB sequential rows are limited by per-request latency. The read takes 87 µs per request (1 441 MiB/s), and the single outstanding request leaves the device idle between transfers. Sections 5 and 7 show throughput climbing once larger blocks or deeper queues keep more data moving.

Two caveats apply. First, a QD=1 random read cannot be prefetched, so this is a genuine floor. Second, the p99.9 column already shows occasional outliers of several hundred microseconds to milliseconds from background flush and garbage collection, which preview the tail behaviour in Section 8.

The sequential-write row is unstable. Its IOPS IQR spans 891–5 298, and its p99 IQR runs to 34 764.7 µs. The raw repetitions are bimodal. The first two ran at 490 and 891 IOPS with p99 of 30 ms to 35 ms; the last three ran at 5100 to 5460 IOPS with p99 of 1.4 ms. The p50 stays near 90 µs in all five, so typical writes were fast throughout, and the slow repetitions are dominated by a few stalls of tens of milliseconds. The point ran directly after the 2 GiB sequential re-fill that precedes the sequential-read case. We read the first two repetitions as waiting for the controller to flush that burst, and the later ones as the unloaded QD=1 write path. The median (5 100 IOPS) therefore describes the recovered state, and the row should not be read as a stable baseline. The full `rerun_clean.sh` repeats it after a rest; the lean re-run reported here does not include it, so this row is still from the original run. `blkbench` now logs each point's start time (`timeline.csv`), so such stalls can be placed in time.

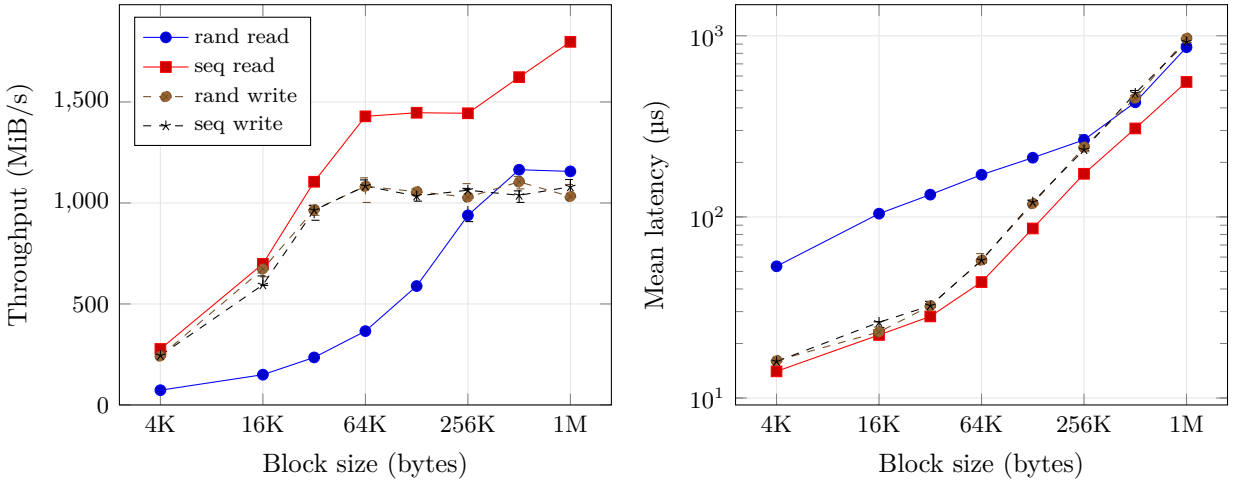
An early pilot run, since discarded, measured this sequential read directly after the random-write case and got only a fraction of the rate above. That discrepancy led to the re-fill step in the procedure, and the `layout` experiment (Section 9.3) reproduces it: 224 vs. 1 443 MiB/s.

5 Block-Size Sweep

Hypothesis (from first principles, then compared with the data)

As the block grows from 4 KiB to 256 KiB at QD=1, small-block I/O should be *IOPS-limited* (throughput = IOPS × block, rising with block size) and large-block I/O should approach a *throughput ceiling* where IOPS falls as 1/block. Sequential should beat random at every block size, with the gap largest at small blocks where per-op overhead dominates.

Fixed	one job; `O_DIRECT`; 5 repetitions of 4 s for reads; 3 repetitions capped at 256 MiB each for writes
Varied	block size 4 KiB–1 MiB (8 sizes); random vs. sequential; 100% read vs. 100% write; QD=1 (all four combinations) and QD=32 (sequential read and write)
Ceiling	PCIe 3.0 ×4: 3756 MiB/s raw (Section 2)
Script	`SSD_Performance_Profiling/scripts/run.sh block_size`



(a) QD=1 throughput.

(b) QD=1 mean latency (same runs).

Figure 1: QD=1 block-size sweep, reads (solid) and writes (dashed); medians with IQR bars. Random-read throughput rises with block size because latency grows more slowly than the transfer size. Sequential reads stay near 1.4 GiB/s from 64 to 256 KiB and reach 1.8 GiB/s at 1 MiB, a single-request limit well below the 3756 MiB/s link. Random and sequential writes both stay at the ∼1.05 GiB/s SLC ingest rate from 64 KiB up; the original run's collapse of the ≥256 KiB random writes did not recur after a rest (see text). Data: the random writes ≥256 KiB from the lean re-run of 2026-10-21; all other points from the 2026-10-21 run.

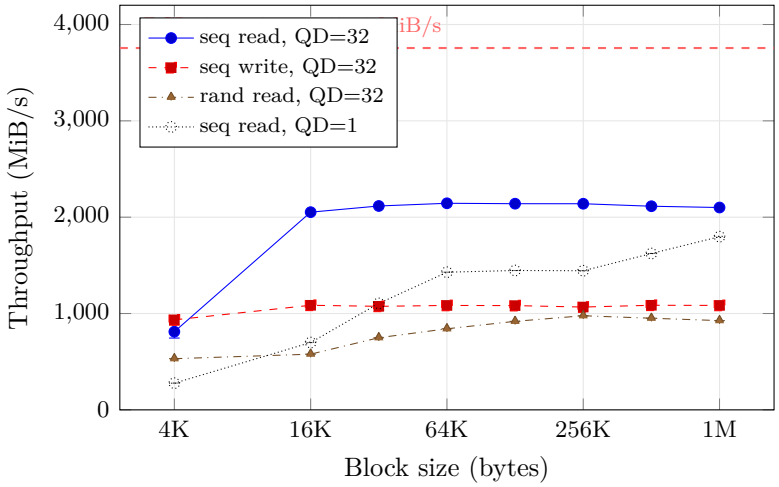


Figure 2: Sequential throughput at QD=32 against the measured-link ceiling (red), with the QD=1 read curve for reference. Deep queues remove the per-request latency limit, so the remaining gap to the line is the device's internal bandwidth plus protocol overhead. Data: the QD=32 random reads from the lean re-run of 2026-10-21; all other points from the 2026-10-21 run.

5.1 Analysis

Random reads at QD=1 are latency-bound, not size-bound. IOPS fall only from 18 711 at 4 KiB to 3 754 at 256 KiB, so throughput climbs from 73 to 939 MiB/s. Per-request latency is not fixed: it grows from 53 to 266 µs (5×) over a 64× range of sizes. A useful decomposition is $W(b) \approx W_0 + t_{\text{xfer}}(b)$: a fixed access cost W_0 of roughly 50 µs (NAND sense plus command overhead, visible at 4 KiB) plus a transfer term. Up to 256 KiB the transfer term grows *sub-linearly* with b, because the controller stripes a request across several dies and planes and reads them in parallel. Beyond that the stripes are exhausted. Latency then doubles with size (429 µs at 512 KiB, 865 µs at 1 MiB), and throughput plateaus at 1 164–1 156 MiB/s.

Sequential reads at QD=1 hit a single-request limit. The 4 KiB sequential read takes only 14.0 µs, versus 53.4 for random. The controller detects the stream and reads ahead, so most requests hit its internal buffer. From 64 KiB to 256 KiB, throughput is flat at 1 429–1 447 MiB/s, and IOPS fall as $1/b$. Larger requests then lift it to 1 798 MiB/s at 1 MiB, because each command covers more of the internal pipeline. Even that is only 48 % of the 3756 MiB/s PCIe 3.0 ×4 link. The QD=1 limit is therefore how fast the controller streams one command at a time, not the interface. The sequential/random gap is widest at 4 KiB and narrows as both patterns become transfer-dominated.

Deep queues do not reach the link ceiling (Figure 2). At QD=32, sequential reads flatten at 2 052–2 144 MiB/s from 16 KiB upward. They reach 2 100 MiB/s at 1 MiB, about 56 % of the raw link rate and well below the 85 % to 90 % that TLP/DLLP overhead would allow. The binding limit is the drive's NAND/controller back end, not PCIe. Sequential writes at QD=32 are flat at 1 084 MiB/s for every block from 16 KiB up. That is the SLC-cache ingest rate: the same ∼1.06 GiB/s appears in the preconditioning passes and in the first seconds of the sustained fill (Section 9).

Random reads at QD=32 (lean re-run) reach 135 967 IOPS at 4 KiB, peak at 977 MiB/s at 256 KiB and give 927 MiB/s at 1 MiB (dash-dotted curve in Figure 2), against 2 100 MiB/s for sequential reads. Even with 32 large requests in flight, random placement costs more than half of the sequential bandwidth. One possible reason, which we did not test, is that the controller's read-ahead and die striping work best on logically sequential access. The 4 KiB point is about 5 % above the QD=32 point of the original queue-depth sweep (129 513 IOPS, Section 7). We take that difference to be the run-to-run variation between a rested drive and one in the middle of a write-heavy sequence.

Writes at QD=1. At 4 KiB, random writes post 61 897 IOPS against 18 711 for reads. The write buffer acknowledges before the NAND program, so write latency reflects the host→buffer transfer, not flash timing. Accordingly, the random and sequential write curves coincide across the sweep (1 083 vs. 1 084 MiB/s at 64 KiB). Both flatten at about 1.05 GiB/s from 64 KiB, the same ingest rate as at QD=32. Unlike reads, writes reach their ceiling with a single request in flight.

A drive-state episode, not a block-size effect. In the original run, the random-write points at 256 KiB, 512 KiB and 1 MiB fell to 64 MiB/s to 66 MiB/s, with p99 latencies of 33 ms to 45 ms. That is the ∼60 MiB/s direct-to-QLC rate the sustained experiment measures once the SLC cache is full. They were the last write points before the sequential-read group, after about 10 GiB of writes in that run, and given how small the cache turned out to be (Section 9), that was plausibly enough to fill it. The lean re-run measured the same three points again after a 5 min rest, with 15 s pauses and a 128 MiB cap per repetition. They ran at 1 029, 1 104 and 1 032 MiB/s, with p99 of 455–1 132 µs, on top of the sequential-write curve, so the earlier collapse reflected cache exhaustion rather than block size.

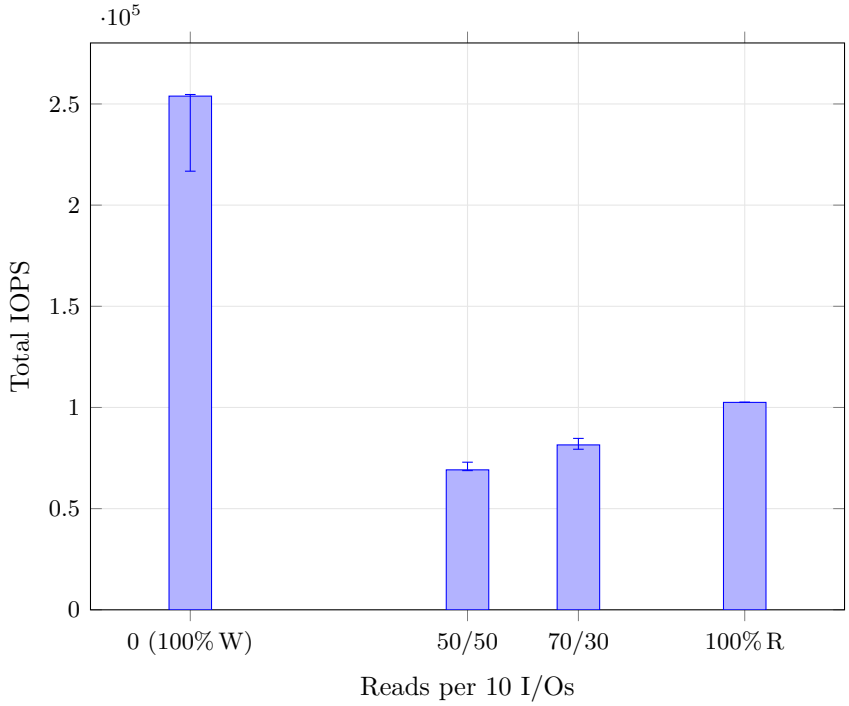


Figure 3: Total 4 KiB random IOPS at QD=16 across read/write mixes (median, IQR bars). Both pure workloads beat the mixes; the 50/50 mix is the worst.

Table 7: Read/write mix at QD=16, 4 KiB random: medians with the IQR in brackets; latencies in µs.

Mix	Total IOPS	Mean latency	p99
100% read	102 514 [102 472, 102 633]	154.1 [153.9, 154.2]	556 [555, 561]
70/30	81 497 [79 349, 84 712]	193.7 [186.4, 199.3]	757 [718, 796]
50/50	69 172 [68 792, 72 950]	229.4 [217.3, 230.8]	989 [962, 1 008]
100% write	253 874 [216 774, 254 678]	62.7 [62.5, 72.8]	79 [69, 528]

6 Read/Write Mix

Hypothesis (from first principles, then compared with the data)

At a fixed block size and queue depth, a mixed read/write workload should deliver *fewer* total IOPS than either pure workload, not a weighted average: reads and writes contend for the controller and flash channels, and a read queued behind a slow program/erase inherits its latency. We expect pure writes to post the highest IOPS at QD=16 (write-buffer absorption) and the 50/50 mix to be the worst.

Fixed	4 KiB random; iodepth 16; one job; `O_DIRECT`; 5 repetitions of 4 s (points with writes also stop at 256 MiB)
Varied	reads per 10 I/Os, deterministically interleaved: 10 (100% R), 7 (70/30), 5 (50/50), 0 (100% W)
Script	`SSD_Performance_Profiling/scripts/run.sh rw_mix`

6.1 Analysis

Pure writes are fastest, at 253 874 IOPS and 63 µs mean, because at QD=16 the controller absorbs them into its SLC/SRAM buffer. Pure reads reach 102 514 IOPS. Mixing is worse than either pure case. The 70/30 mix drops to 81 497 IOPS and the 50/50 mix to 69 172, below even the read-only number. A write landing mid-stream occupies a die (program, or the buffer flush it triggers), and reads mapped to that die queue behind it. The interleaved writes also break up the write buffer's batching.

The p99 latency gives a related but not monotonic picture. It rises from 556 µs for pure reads to 757 (70/30) and 989 µs (50/50). Pure writes break the trend: their median p99 of 79 µs is the lowest of all four mixes, but its IQR (69–528 µs) is by far the widest. A pure-write repetition either stays inside the write buffer and sees no tail, or it meets a buffer flush and sees a read-like tail. The worst *typical* tail belongs to the 50/50 mix, not to the largest write fraction, so p99 is not monotonic in the write fraction.

The lesson: a "70% read" workload is not 0.7 of the read result plus 0.3 of the write result. The interaction term is large and negative, which is why the spec requires measuring the mixes rather than interpolating them.

7 Queue-Depth Sweep

Hypothesis (from first principles, then compared with the data)

IOPS should rise steeply with queue depth as the SSD's parallel NAND channels fill, then saturate at a knee beyond which latency grows roughly linearly with queue depth for no extra throughput. By Little's Law the number of outstanding I/Os $L = \lambda W$ should track the configured iodepth while the queue stays full; where it stops tracking reveals a host-side or engine limit.

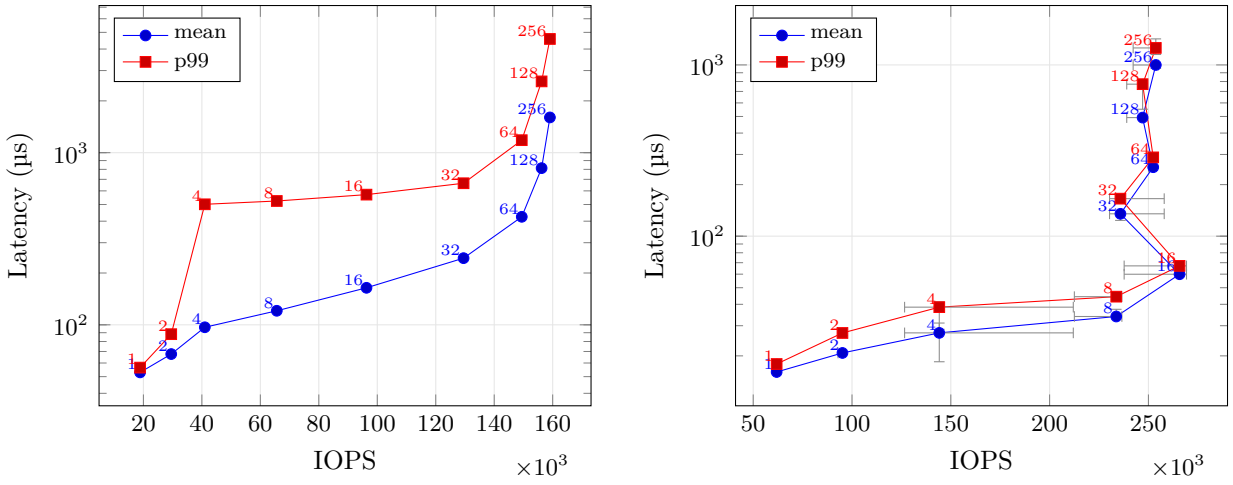
Fixed	4 KiB random; one job; `O_DIRECT`; 5 repetitions of 4 s (writes also stop at 256 MiB)
Varied	iodepth 1, 2, 4, …, 256; 100% read and 100% write
Knee rule	the smallest QD whose doubling raises median IOPS by less than 10 % (computed by `run.sh` into `queue_depth_knee.csv`)
Effective QD	checked in-band with Little's Law (Table 8); no `iostat` available
Script	`SSD_Performance_Profiling/scripts/run.sh queue_depth`

7.1 Knee

We define the knee as *the smallest QD whose doubling raises median IOPS by less than* 10 %. Past that point, most of any extra concurrency becomes added latency rather than throughput.

Reads. The rule gives QD 64 (149 422 IOPS, 424 µs mean). Doubling to QD 128 adds only 4.5 % IOPS. The preceding doubling, QD 32→64, still gained 15 %. The grid doubles at each step, so the knee lies between 32 and 64.

Writes. The write points at QD 4–32 come from the lean re-run, each after a 15 s pause; QD 1, 2 and 64–256 are from the original run. The rule gives QD 16 (265 754 IOPS, 59.9 µs mean), and doubling to QD 32 changes IOPS by −11.3 %. Write IOPS rise from 61 860 (QD=1) through 144 161 (QD=4) and 233 722 (QD=8) to the QD=16 peak, then stay at 235 830–253 743 out to QD=256. The dip at QD=32 is not significant: its IQR (230 331–257 997) overlaps that of QD=16 (237 723–269 281). The QD=4 point is the least stable (IQR 126 613–211 985). In the original run the QD=8 write point caught a buffer-flush stall (16 884 IOPS, p99 27.9 ms). The knee rule then returned QD=4, and the text had to override it. After the pauses of the lean re-run, the stall did



(a) 4 KiB random read.

(b) 4 KiB random write.

Figure 4: Throughput vs. latency as queue depth grows; each point is labelled with its QD. Bars show the IQR across repetitions in both IOPS and latency. Read IOPS reach 149 422 at the knee (QD 64) and at most 159 047 at QD=256; deeper queues then add only latency. Data: the write points at QD 4–32 from the lean re-run of 2026-10-21; all other points from the 2026-10-21 run.

not recur, and the rule's answer agrees with the shape of the curve. The write plateau of ∼250k IOPS × 4 KiB ≈ 0.97 GiB/s matches the SLC ingest rate measured in Section 5. Writes saturate the buffer's intake, not the NAND parallelism that limits reads.

7.2 Little's Law as a consistency check

Little's Law [2] relates the mean number of requests in a system L, the throughput λ and the mean time in system W: $L = \lambda W$. In a *closed-loop* harness it holds *by construction*. There are exactly QD request slots, and each completion is resubmitted immediately, so $\lambda W = \mathrm{QD} \cdot W/(W+g)$, where g is the host-side gap between reaping a completion and resubmitting it. Matching QD therefore does not show that the device behaves in a particular way. It shows that the timers are consistent, that no slot is lost, and that the host keeps up ($g \ll W$). A ratio L/QD clearly below 1 would mean the issuing thread, not the device, is the bottleneck. On a tmpfs control, where I/O completes synchronously inside `io_submit`, `make test` measures $L/\mathrm{QD} \approx 0.57$ at QD=8. Note also that W is measured from submission, so L counts requests in the kernel block layer and driver as well as in the device; the true device-queue occupancy can only be lower.

Inference: internal parallelism. Little's Law can also be applied to the *device*, as an inference rather than a measurement. At saturation the drive completes $\lambda_{\max} \approx 159\,047$ reads/s. If an isolated 4 KiB read occupies the flash back end for roughly its unloaded service time S, the mean number of reads *in service* is $L_{\mathrm{busy}} = \lambda_{\max} S$. Taking S between 40 µs (the QD=1 latency minus ∼10 µs of host and transfer overhead) and 53 µs (the whole QD=1 latency) gives $L_{\mathrm{busy}} \approx 6$–$8$. So of the 64–256 requests outstanding past the knee, only about 6–8 are being served by the NAND at any moment, and the rest wait in queues. That is the queueing-delay component of the latency growth in Figure 4. Micron does not publish the die or channel count for the 2210. A 512 GB drive built from 1-Tbit QLC dies would have 4 dies, in which case 6–8 concurrent reads would mean about two reads in flight per die (multi-plane or independent-plane reads). This is a plausibility argument

Table 8: Little's Law per queue depth: λ (IOPS) and W (mean latency, µs) are medians with the IQR in brackets; $L = \lambda W$ is computed per repetition and then summarized. `4 KiB` random.

Mode	QD	λ (IOPS) [IQR]	W [IQR]	$L = \lambda W$ [IQR]	L/QD
read	1	18 866 [18 854, 18 885]	52.9 [52.9, 53.0]	1.00 [1.00, 1.00]	0.998
read	2	29 518 [29 496, 29 525]	67.6 [67.6, 67.7]	2.00 [2.00, 2.00]	0.998
read	4	41 006 [40 991, 41 072]	96.8 [96.6, 96.8]	3.97 [3.97, 3.97]	0.992
read	8	65 629 [64 935, 65 709]	120.7 [120.5, 122.0]	7.92 [7.92, 7.92]	0.99
read	16	96 303 [96 269, 96 406]	164.1 [163.9, 164.2]	15.80 [15.80, 15.80]	0.988
read	32	129 513 [129 050, 129 607]	243.9 [243.8, 244.4]	31.59 [31.54, 31.60]	0.987
read	64	149 422 [149 292, 149 458]	424.2 [424.2, 424.5]	63.38 [63.38, 63.39]	0.99
read	128	156 211 [156 149, 156 431]	813.4 [812.2, 813.6]	127.05 [127.04, 127.06]	0.993
read	256	159 047 [158 980, 159 094]	1 602.6 [1 602.2, 1 603.3]	254.89 [254.89, 254.89]	0.996
write	1	61 860 [61 800, 62 223]	16.1 [16.0, 16.1]	1.00 [1.00, 1.00]	0.996
write	2	95 161 [92 914, 95 226]	20.8 [20.8, 21.3]	1.98 [1.98, 1.98]	0.989
write	4	144 161 [126 613, 211 985]	27.2 [18.5, 31.1]	3.93 [3.92, 3.94]	0.982
write	8	233 722 [212 524, 236 579]	34.0 [33.6, 37.4]	7.95 [7.94, 7.95]	0.993
write	16	265 754 [237 723, 269 281]	59.9 [59.1, 67.0]	15.92 [15.92, 15.93]	0.995
write	32	235 830 [230 331, 257 997]	135.1 [123.6, 138.4]	31.88 [31.87, 31.88]	0.996
write	64	252 431 [250 115, 252 712]	252.8 [252.6, 255.2]	63.83 [63.80, 63.83]	0.997
write	128	247 061 [239 034, 247 254]	492.5 [491.5, 502.3]	121.77 [121.44, 127.54]	0.951
write	256	253 743 [242 276, 255 085]	999.1 [956.0, 1 003.7]	254.67 [254.67, 254.86]	0.995

with an assumed S, not a measured die count.

7.3 Analysis

Read IOPS climb from 18 866 at QD=1 to 96 303 at QD=16 and 129 513 at QD=32, then flatten at 149 422–159 047. Scaling is sub-linear from the start. Successive doublings from QD=1 add 56, 39, 60, 47, 34, 15, 4.5 and 1.8 %, with no distinct regime of linear scaling. The pilot run's impression that QD 1→4 scaled unusually poorly did not reproduce: here it scaled 2.2×. Beyond the knee, mean latency doubles with each doubling of QD (424→813→1 603 µs), which is the textbook saturation signature. With λ flat, $L = \lambda W$ can follow QD only if W grows in proportion, so every added request simply waits.

For reads, L/QD stays between 0.987 and 0.998 at every depth up to 256 (Table 8). The single `libaio` thread keeps the queue full, and the saturation belongs to the device. For writes the ratio is 0.95–0.997. The lowest value, 0.95 at QD=128, coincides with the widest p99.9 there (16 712 µs). We read it as completions bunching around a flush stall rather than a host limit, but with five repetitions we cannot separate the two.

The practical reading for reads: QD ≈ 32–64 extracts nearly all the throughput at a quarter to half a millisecond, and deeper queues only buy latency. Writes need only QD ≈ 16, as long as the SLC cache has room. Once it does not, write throughput is set by QLC programming, whatever the queue depth (Section 9).

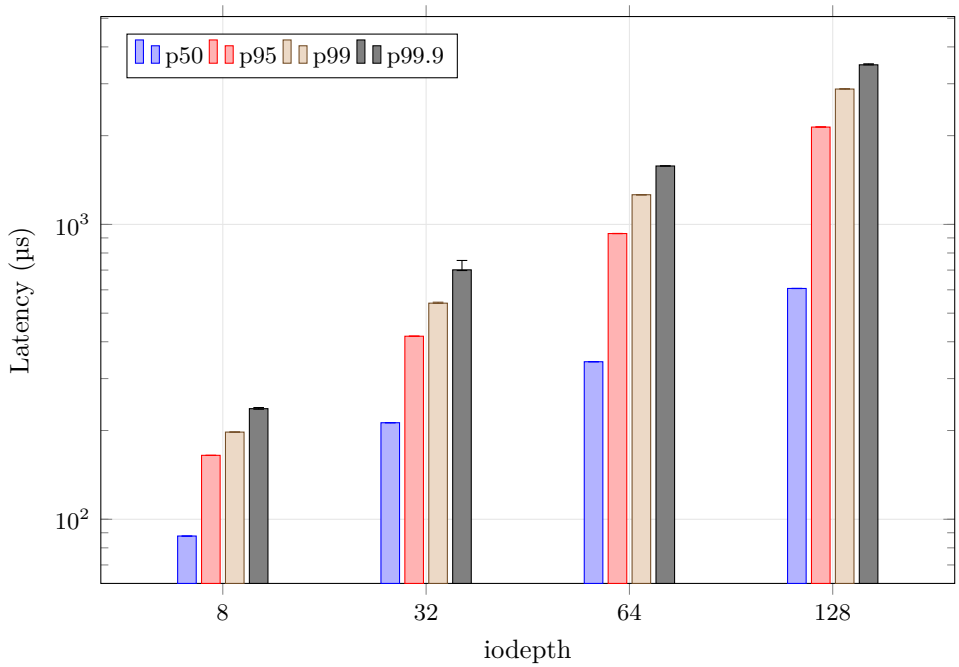


Figure 5: Latency percentiles at a mid queue depth (8), around the knee (32, 64) and past it (128); medians with IQR bars (data: lean re-run of 2026-10-21). Measured after a 10 min rest on a freshly preconditioned file. The upper tail widens faster than the median as QD grows: p99.9/p50 rises from 2.7× at QD=8 to 5.7× at QD=128.

8 Tail Latency

Hypothesis (from first principles, then compared with the data)

Moving from a mid queue depth to the near-knee depth should raise the median modestly but stretch the upper percentiles much more: deeper queues mean more requests waiting behind the occasional slow flash operation, so the p99/p99.9 gap over p50 should widen.

Fixed	4 KiB random read; one job; 6 s runs, 5 repetitions; NVMe temperature recorded per repetition
Varied	iodepth 8 (mid), 32 and 64 (bracketing the knee of Section 7), 128 (past it)
Drive state	lean re-run: after the short sustained fill, the large file was deleted, the drive rested 10 min, and a fresh 4 GiB test file was preconditioned sequentially; reads cover its first 2 GiB
Script	`SSD_Performance_Profiling/scripts/run.sh tail` (inside `rerun_clean.sh lean`)

8.1 Analysis

On a rested drive the tail widens with queue depth roughly as hypothesised, but modestly:

- QD=8: p50 88, p99 198, p99.9 237 µs; p99.9/p50 = 2.7×.
- QD=32: p50 213, p99 541, p99.9 701 µs; 3.3×.
- QD=64: p50 342, p99.9 1 578 µs; 4.6×.
- QD=128: p50 607, p99.9 3 478 µs; 5.7×.

From QD=8 to 32 the median grows 2.4× and p99.9 3.0×; from 32 to 64, 1.6× and 2.3×. IOPS meanwhile barely grow past the knee (152 073→157 257). Past the knee every added request waits in a queue, and a request queued behind one of the drive's occasional slow reads waits longer. That

stretches the upper percentiles more than the median. The p99/p50 ratio follows the same trend (2.3× at QD=8 to 4.7× at QD=128).

These tail runs agree with the original queue-depth read sweep wherever both measured the same depth: 135 794 vs. 129 513 IOPS at QD=32, and 157 257 vs. 156 211 at QD=128. The exception is QD=8, where the rested drive is faster (80 005 vs. 65 629 IOPS) and has a much lower p99 (198 vs. 524 µs). The original sweep's p99 sat near 0.5 ms at every depth from 4 to 16. We suspect that it, too, met some background flash work left by the earlier write experiments, but we did not test this.

The tail depends on recent writes. The original tail run (`tail/20261021-103201`) started immediately after the queue-depth *write* sweep, which wrote ∼11 GiB of random 4 KiB data, with no idle gap. At QD=32 it delivered 64 475 IOPS, half the rested value, at a median of 385 µs, and its p99.9/p50 ratio rose from 4.1× at QD=8 to 24× at QD=128 (p99.9 16.0 ms). Its temperature, 50.9 °C to 52.9 °C, was no higher than that of the lean tail run (39.9 °C to 52.9 °C per-point medians), so the difference is not thermal. The two runs differ in more than drive state, including the file size (2 GiB vs. 4 GiB) and the time of day. The likely cause is still foreground reads competing with the background folding and garbage collection of freshly written data. A read that arrives during a program, erase or GC burst waits milliseconds, and deeper queues hold more requests behind each such stall. The tail ratio is therefore a sensitive detector of background flash activity. Queueing alone keeps it at 3–6×, while a recent heavy write load pushes it to 24×.

These are the numbers that govern provisioning for a latency-SLA service, not the mean. At QD=32 one read in a thousand takes 701 µs on a rested drive, but took 2.6 ms right after heavy writes. The knee from Section 7 is therefore a latency decision as much as a throughput one: past it, almost no IOPS are gained and these tails inflate fastest. Over both runs the NVMe composite temperature stayed at 31.9 °C to 61.9 °C, far below the 86.85 °C warning threshold (Section 9).

9 Device State and Working Set

Hypothesis (from first principles, then compared with the data)

Back-to-back sustained sequential writes should first ride the SLC/DRAM write cache (a burst), then settle to a lower steady-state throughput once the cache must flush to QLC in the background. Occasional latency spikes from garbage collection should appear in the upper tail even while mean throughput is steady.

Hypothesis (from first principles, then compared with the data)

(Added before the large-file runs.) (i) A single sequential write far larger than the SLC cache should show a sharp *cliff*: burst throughput near the QD=32 write rate of Section 5, then a drop to the native QLC program rate, several times lower. (ii) After that, random 4 KiB writes reach a lower steady state than in the burst, with heavier tails from garbage collection. (iii) A short idle period lets the controller drain the cache and recover part of the burst rate. (iv) Random reads should slow as the working set grows from 1 GiB to 60 GiB, because more logical-to-physical map lookups miss the controller's mapping cache. (v) "Sequential" reads of data written randomly should be much slower than of data written sequentially. (vi) Temperature should rise during sustained writes but stay below the 86.85 °C warning threshold reported in sysfs (`temp1_max`).

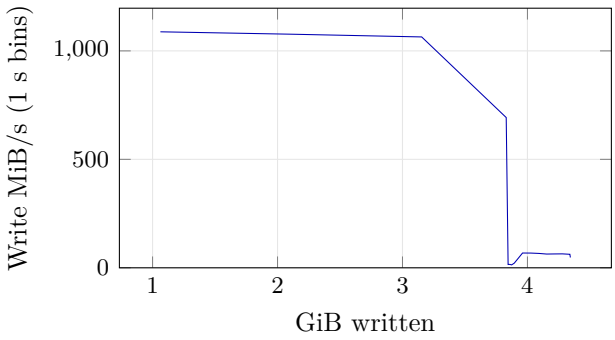
Table 9: Sustained-write summary (computed by `run.sh` into `sustained_summary.csv`). Burst is the median of the leading seconds that stay above half the first second's rate. The cliff is the first second, starting a 5-second window, whose rate falls below half the burst; the table gives the volume written before it (data: lean re-run of 2026-10-21). Steady is the median of all seconds after the cliff. The lean re-run stops the fill 10 seconds after the cliff; without a cliff the fill volume would be a lower bound on the cache size. The random phase mixes two states, so the table gives its byte-weighted average and the per-second quartiles. The recovery probe reached its 1 GiB cap within its first second, so its first and last 5 s are the same sample.

Burst sequential write (MiB/s) / duration (s)	1 071 / 4
Cliff after (GiB written / s)	3.8 / 4
Post-cliff steady sequential write (MiB/s)	63
Steady / burst	0.059
Fill: GiB, seconds, average MiB/s	4.3, 15, 295
Random 4 KiB write, QD32: phase average IOPS	30 090
per-second IOPS quartiles; median per-second p99 (µs)	14 306–15 343; 35 726
Recovery after 60 s idle: first / last 5 s (MiB/s)	1 104 / 1 104
NVMe temperature min–max (°C)	31.9–40.9

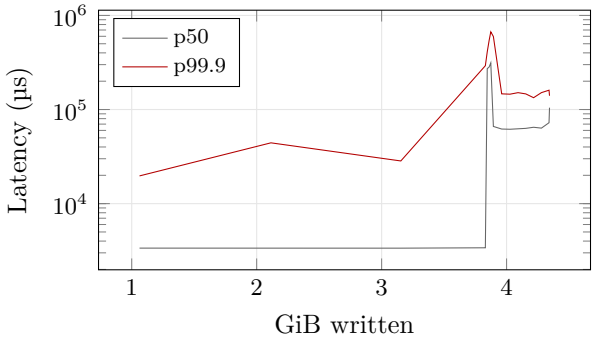
`sustained`	lean re-run: one O_DIRECT sequential pass (128 KiB, QD32) over a fresh `fallocate`d file of up to 40 GiB, stopped 10 seconds after the throughput stays below 30 % of the first-5-second burst median (`STOP_BELOW=0.3`, `STOP_HOLD=10`). Then 4 KiB random writes at QD32 over the written part for 60 s or 1 GiB, a 60 s idle gap, and a 10 s / 1 GiB sequential recovery probe. Throughput, p50/p99/p99.9/max latency and NVMe temperature are logged every second
`working_set`	lean re-run, on the freshly preconditioned 4 GiB test file right after `tail`: 4 KiB random read (QD1, QD32) and random write (QD32, 64 MiB cap) restricted to its first 1, 2 and 4 GiB, in the order 1, 2, 4, 4, 2, 1 GiB with 2 repetitions per pass
`layout`	original run, on the 2 GiB test file: 128 KiB sequential read (QD1, QD32) and 4 KiB random read, first after a sequential fill, then after a 4 GiB random 4 KiB overwrite (about 86 % of blocks rewritten in random order)
Volume / safety	the fill file is capped at 40 GiB (77 GiB free; the gate needs file + 10 GiB) and is deleted before the next step. The filesystem has no online discard
Drive state	`sudo fstrim -v /` at 21:34:54, then at least 30 min idle; the script's idle check passed and the fill started at 22:06:25. The scratch partition (`nvme0n1p3`, 124.9 GiB) had 77 GiB free; the drive's other partitions, including a 351 GiB one this study never touches, were not trimmed
Script	`rerun_clean.sh lean` (`sustained`, then `tail`, then `working_set`); `run.sh layout` (inside `all`)

9.1 SLC-cache exhaustion and steady state

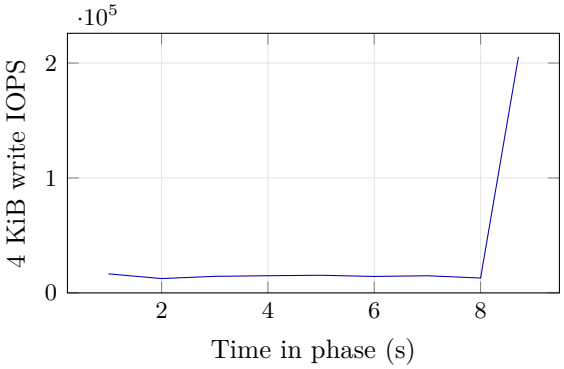
Analysis. The fill began at the SLC ingest rate, 1 071 MiB/s, for 4 s. After 3.8 GiB it collapsed. Four seconds at 13 MiB/s to 21 MiB/s, with per-second median latencies of up to 0.31 s at QD32, were followed by seven seconds at 62 MiB/s to 68 MiB/s. The stop rule then ended the fill at 4.3 GiB after 15 s (average 295 MiB/s). The median of all post-cliff seconds is 63 MiB/s, 0.059 of the burst. A burst 17× the steady rate is far more than the "several times" of hypothesis (i). The steady value rests on only 11 s of data. It matches, however, the original run, whose 60 GiB fill ran mostly at 60 MiB/s to 64 MiB/s (with dips to 28 MiB/s to 46 MiB/s) for twenty minutes after its cliff (`sustained/20261021-105439`). We therefore take it as the rate at which this QLC drive



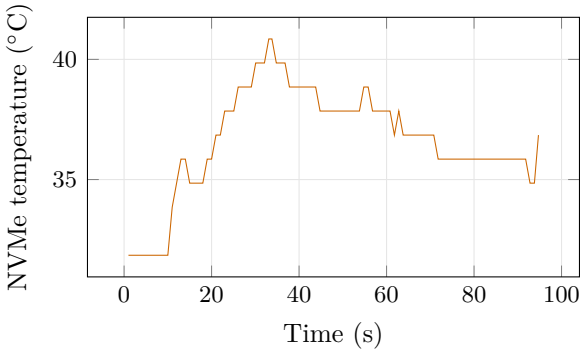
(a) Sequential fill: throughput vs. volume.



(b) Sequential fill: per-second latency.



(c) Random-write phase (QD32).



(d) Composite temperature, all phases.

Figure 6: Sustained-write experiment, one-second samples (data: lean re-run of 2026-10-21). (a) The SLC-cache cliff appears as the drop in throughput after 3.8 GiB; the stop rule ended the fill at 4.3 GiB. (b) The latency during the fill. (c) The random-write phase after the fill; its last, partial sample (0.7 s) jumped to SLC speed and reached the 1 GiB cap. (d) Temperature across the baseline idle, fill, random, idle and recovery phases.

writes when it must program QLC directly while also folding cached data.

How large is the cache? Hypothesis (i) assumed a fill "far larger than the SLC cache", and the lean plan sized the fill on the assumption that a fresh cache would hold ∼20 GiB. Instead, on a drive that had been trimmed and idle for half an hour, the cliff came at 3.8 GiB. That is a measurement of the SLC capacity *available to this fill in this drive state*, not of a fixed property of the drive, and it falls well short of the expectation. The original run had found a similar cliff at 3.1 GiB, and we had attributed that small value to the run order, since that fill followed ∼60 GiB of writes with no idle and no TRIM. The lean re-run removes those confounds and adds only 0.7 GiB, so the run order was not the main reason. A plausible explanation, which we did not test, is a dynamic SLC cache sized by free space, a common design in QLC client drives. Such a controller can only borrow blocks it believes are unused. The scratch filesystem had 77 GiB free and was trimmed, but the rest of the drive, including a 351 GiB partition this study never touches, was not, so the drive-wide free space that such a cache would scale with is unknown and may be small. Two other explanations remain open. Thirty minutes may not have drained all folded data after the TRIM, or the 2210 may simply use a small cache in this capacity class. Telling these apart would need fills at several known free-space levels, or the vendor and SMART counters that need root (the lean run's SMART snapshot failed without it). Under the free-space explanation the cache size is state dependent, and 3.8 GiB describes this drive at its current fill level.

The random-write phase wrote 4 KiB blocks at QD32 over the 4.3 GiB just written. For eight seconds the per-second rate stayed at 12 500 to 16 500 IOPS (median 14 924; quartiles 14 306–15 343), with a median per-second p99 of 35.7 ms. That is about 16× below the in-cache random-write rate at the same depth (235 830 IOPS, Section 7), and it supports hypothesis (ii). In the final 0.7 s the rate jumped to ∼205k IOPS with a p99 of 149 µs, as if the controller had just freed SLC blocks in the background. That burst wrote 56 % of the phase's 1 GiB cap, which ended the phase after 8.7 s. The byte-weighted phase average in Table 9, 30 090 IOPS, therefore mixes two states and should not be read as a steady state. The per-second median is the better summary.

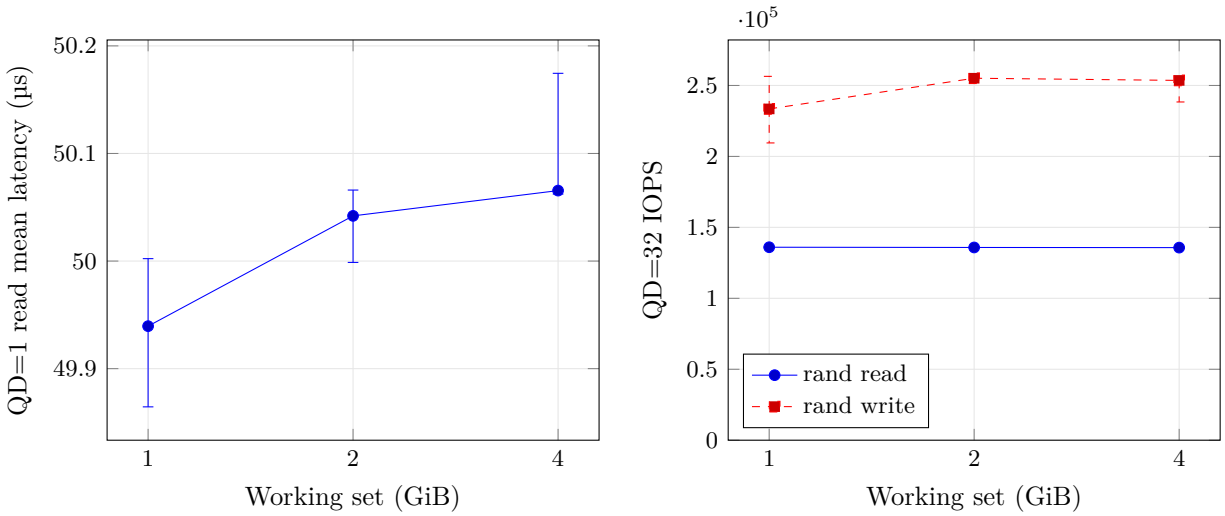
After the 60 s idle, the recovery probe wrote its whole 1 GiB at 1 104 MiB/s, the full burst rate, in under a second. At least 1 GiB of cache was therefore free again, and hypothesis (iii) holds at this scale. The cap means we cannot say how much of the 3.8 GiB came back. This reverses the original run, where the same idle followed tens of gigabytes of writes and gave no recovery (61 then 19 MiB/s). Recovery depends on the size of the fold backlog, not on the idle time alone.

Two caveats. First, because the cliff came early, the lean fill wrote only 4.3 GiB of a 512 GB drive, and its post-cliff and random phases last seconds rather than minutes. This is the short-term behaviour after the cache fills, not the SNIA full-drive steady state, which would need the raw device (ruled out by the data-safety rule). Second, a TRIM only marks blocks free; it does not make the drive fresh, and the state of the untrimmed rest of the drive is unknown.

Temperature. The composite temperature ranged from 31.9 to 40.9 °C during this experiment, and from 31.9 °C to 61.9 °C over both runs (the peak was during the QD=32 sequential-read sweep of the original run). Both are far below the 86.85 °C warning and 89.85 °C critical thresholds reported by the drive. The throughput fell within one second at 35.9 °C and stayed low while the temperature moved only between 34.9 and 37.9 °C, so the drop comes from the cache, not from thermal throttling. In the original run the same cliff came at 50.9 °C.

9.2 Working set (LBA range)

QD=1 random-read latency is flat: 49.9, 50.0 and 50.1 µs for spans of 1, 2 and 4 GiB. So are QD=32 read IOPS (135 997, 135 831 and 135 727), which differ by less than 0.3 %. QD=32 random writes



(a) QD=1 random read latency.

(b) QD=32 random read and write IOPS.

Figure 7: 4 KiB random I/O restricted to the first 1, 2 and 4 GiB of a freshly preconditioned 4 GiB test file (medians of 4 repetitions, 2 in each of a forward and a reverse pass, with IQR bars; data: lean re-run of 2026-10-21). No span effect is visible in this range.

Table 10: The same reads on the same 2 GiB file after a sequential fill and after a random 4 KiB overwrite (medians of 5 repetitions).

Read	after sequential fill	after random overwrite
128 KiB seq, QD=1 (MiB/s)	1 443	224
128 KiB seq, QD=32 (MiB/s)	2 093	229
4 KiB rand, QD=1 (IOPS)	18 803	18 747

give 233 476, 255 095 and 253 564 IOPS, all at the in-cache write rate of Section 7. The lower 1 GiB median comes with the widest IQR (209 505–256 425) and does not indicate a trend. Because every span ran once on the way up and once on the way down, a drift over time would have shown up as a forward/reverse difference. For reads the two passes agree within 1 %. The absolute read latency is about 6 % lower than the original run's clean baseline on the 2 GiB file (53.1 µs QD=1 random read in `layout`).

Hypothesis (iv) predicted slower random reads as the span grows from 1 to 60 GiB, because map lookups would miss the controller's mapping cache more often. The lean re-run covers only 1–4 GiB. At 4 bytes of map per 4 KiB page, 4 GiB needs only 4 MiB of map, so the flat result is consistent with a mapping cache that covers at least that much, and says nothing about larger spans. The original run did sweep 1 to 60 GiB (`working_set/20261021-105439`). There, QD=1 latency rose by about 37 % and QD=32 read IOPS fell by 25 %. That sweep ran right after the 60 GiB fill, with tens of gigabytes still being folded, in increasing span order only, and even its 1 GiB point reached only 12 100 QD=1 IOPS against 20 000 here. Its trend cannot be separated from the fold backlog and elapsed time, so hypothesis (iv) remains untested beyond 4 GiB. We could not tell whether the drive uses a host memory buffer (no allocation message in the kernel log).

Table 11: Same 4 KiB sequential-read workload, direct vs. buffered: medians with the IQR in brackets. Only the `O_DIRECT` flag differs.

Setup	IOPS	MiB/s	Mean (µs)	p99 (µs)
`O_DIRECT` (correct)	70 920	277	14.03	18.1
buffered (page cache)	1 334 404	5 213	0.68	1.15

9.3 Data layout: why preconditioning order matters

An early pilot run, since discarded, exposed this effect by accident: a 128 KiB QD=1 sequential read run right after a random-write test was several times slower than the block-size sweep. An SSD places data in the order it was *written*, not by logical address. Table 10 tests this directly, and the hypothesis is confirmed strongly. After the random overwrite, QD=1 sequential reads drop from 1 443 to 224 MiB/s (6.4×). 4 KiB random reads are unaffected (18 803 vs. 18 747 IOPS), since a single page costs the same wherever it sits.

Our sub-prediction that QD=32 would hide the gap was *wrong*. At QD=32 the fragmented file gives 229 MiB/s against 2 093, a 9.2× gap, and no better than at QD=1. Each fragmented 128 KiB request becomes up to 32 scattered page reads. The drive completes ∼1830 such requests per second (about 58k page reads/s) at either depth, well below the 130k random 4 KiB reads/s it sustains at QD=32. We hypothesise that the controller serializes the fragments of one command, or that read-ahead actively wastes bandwidth on the wrong pages, but we did not test this.

This is the "insufficient preconditioning" pitfall in a subtler form. A sequential benchmark is meaningful only if the data's write history is controlled. That is why `run.sh` rewrites the test file sequentially before every sequential-read group. One group missed this in the original run: in `all`, `fio_check` followed `layout` without a rewrite, so its sequential points measured the fragmented layout (Section 3.1). The script has since been fixed.

10 Misleading Setup

Hypothesis (from first principles, then compared with the data)

Dropping `O_DIRECT` lets the OS page cache serve repeated reads of the same region from RAM after the first pass, so a buffered benchmark will report throughput and latency that describe DRAM, not the SSD—orders of magnitude "better" than the real device, and entirely fictitious as a storage result.

Pitfall	Buffered I/O measuring the page cache instead of the device
Wrong setup	4 KiB sequential reads over a 64 MiB region, *no* `O_DIRECT`
Right setup	Identical workload *with* `O_DIRECT` (page cache bypassed)
Fixed	QD=1, 5 repetitions of 4 s; test file rewritten sequentially just before
Script	`SSD_Performance_Profiling/scripts/run.sh pitfall`

10.1 Explanation

The buffered run reports 1 334 404 IOPS at 5 213 MiB/s and sub-microsecond latency, a 19× inflation over the correct 70 920 IOPS. None of it is a storage measurement. After the first pass the 64 MiB region lives entirely in the page cache, so every later read is a `memcpy` from RAM. The MiB/s figure is memory-copy bandwidth, and the latency is a cache hit, not an NVMe access.

This is the easiest way to publish an impossibly fast SSD number, and an AI-generated benchmark falls into it readily because omitting `O_DIRECT` is the *default* behaviour of `open()`. Three cheap detectors flag it:

1. The throughput exceeds the 3756 MiB/s PCIe 3.0 ×4 link ceiling measured from sysfs (Section 2). No device result can do that.
2. Sub-microsecond "storage" latency is physically impossible for NAND flash.
3. The number depends on the *region size*: it collapses once the region exceeds free RAM, which a true device result must not.

The methodology avoids the trap by construction. `O_DIRECT` is the default for every real run, and the buffered path exists only to produce this comparison. `make test` also asserts that buffered re-reads are more than 5× faster than direct ones on the scratch filesystem, which proves the cache is actually bypassed. Two related traps are covered elsewhere. A wrong effective queue depth is caught by L/QD in Table 8. Insufficient or wrong-order preconditioning, which cut a "sequential" read to a sixth of its true speed in our pilot, is covered in Section 9.3.

11 Anomalies and Limitations

11.1 Unexpected results

- **The SLC cache is small.** On a trimmed drive that had been idle for half an hour, a sequential fill hit the cliff after 3.8 GiB, not the ∼20 GiB we expected. The original run's 3.1 GiB, which we had blamed on run order, was therefore mostly not a run-order artifact. Hypothesis: a dynamic cache sized by drive-wide free space, which is unknown here because only the scratch partition was trimmed (Section 9). Not tested.
- **Random writes beat random reads, until the cache fills.** At QD=1, 4 KiB writes take 16.0 µs and reads 53.2 µs. At QD=16, pure writes (253 874 IOPS) beat pure reads (102 514). The controller acknowledges writes from its SLC/SRAM buffer. The sustained run confirms the mechanism: once the buffer is full, random writes at QD32 drop to about 15k IOPS per second, far below reads.
- **Write collapses inside ordinary sweeps were drive state.** In the original run, three block-size random-write points (≥256 KiB) and the QD=8 queue-depth write point fell to the direct-to-QLC rate, with p99 of tens of milliseconds. The lean re-run measured them again after rests and pauses, and all ran at the in-cache rate (Sections 5 and 7). The knee rule no longer needs overriding for writes.
- **Sequential-read speed depends on write history** (6.4× at QD=1 and 9.2× at QD=32; Section 9.3). Deep queues did not hide the gap, contrary to our prediction.
- **Tails depend on recent writes.** The original tail runs, right after the queue-depth write sweep, had half the IOPS of the rested lean re-run at QD=32 and p99.9/p50 ratios up to 24×, against at most 5.7× on the rested drive (Section 8).
- **FIO and `blkbench` differ on write points by up to** 12 %, in both directions, and the QD=32 ratio changed sign between the two runs (0.78, then 1.12). The alternating order shows that for the sequential write the position within a pair moves a result more than the engine does. A small extra per-I/O cost in FIO at QD=1 cannot be ruled out (Section 3.1).

- **Recovery depends on the backlog.** After a 60 s idle, the lean re-run's recovery probe wrote 1 GiB at full burst speed; the original run, after tens of gigabytes of writes, recovered nothing.

11.2 Limitations

- **Two runs are combined.** The lean re-run (`20261021-220614`) re-measured only the experiments that the original run order had confounded: sustained, tail, working set, the QD=32 random reads and ≥256 KiB random writes of the block-size sweep, write QD 4–32 and the FIO write points. Zero-queue, read/write mix, the read queue-depth sweep, layout, pitfall and the rest of the block-size sweep still come from `20261021-103201`, on a drive that had absorbed earlier writes without a rest. Where both runs measured comparable points, the rested drive was about 5 % faster for 4 KiB reads at QD=32 and 22 % faster at QD=8 (Section 8), so the original-run read numbers may be slightly pessimistic.

- **The lean fill is short.** The cliff came early, so the fill wrote only 4.3 GiB. The post-cliff rate rests on 11 s, the random phase on 8.7 s (its average is skewed by a final SLC-speed burst), and the recovery probe was capped at 1 GiB. The cache size was measured in one drive state only, and the free-space explanation for it is untested.

- **Working set only up to** 4 GiB **on a clean drive.** The lean re-run found no span effect from 1 to 4 GiB; the original 1–60 GiB sweep is confounded by the fold backlog, so larger spans remain untested.

- **Primary engine is not FIO.** The sweeps use `blkbench`. FIO cross-checks it on seven points (Section 3.1): reads agree within 2 %, writes within 12 %, with drive state the main source of the difference. `ENGINE=fio` reruns every sweep through FIO.

- **Closed-loop load only.** Every experiment holds a fixed QD. An open-loop (fixed arrival rate) generator would show queueing delay against offered load, and its tails would differ.

- **No `iostat/perf` cross-check.** Effective queue depth is checked in-band through L/QD (Table 8). That includes time in the kernel block layer, so it bounds device-queue occupancy from above.

- **SMART needs root.** Temperature comes from the NVMe hwmon sensor, which needs no root. The lean re-run's `nvme smart-log` snapshots failed with "Permission denied", so data-units-written, media errors and the throttle counters are unavailable, and throttling is inferred only from temperature and throughput together.

- **Filesystem, not raw device; partial LBA range.** I/O goes to disposable files on ext4: a 2 GiB (original) or 4 GiB (lean) file for the sweeps, a 60 GiB file for the original device-state run (about 13 % of the drive) and 4.3 GiB of fill in the lean one. Full-drive SNIA steady state is out of scope, and only the scratch partition could be trimmed.

- **Byte caps and few repetitions on write points.** Write points stop after 256 MiB (128 MiB in the lean re-run) per repetition and use 3–5 repetitions, so a single flush stall can move a median, as the original QD=8 write point showed. The FIO order split suggests a ∼15 % drive-state uncertainty on write numbers.

- **One `fio_check` group was not re-filled.** Its sequential points measured the fragmented layout left by `layout`. This does not affect the engine comparison, and the script has since been fixed.
- **Fixed data pattern.** All writes use an incompressible pattern. Compressibility was not varied (`blkbench -fill zero` exists for that). One drive, one firmware (`P6MA001`).
- **Report build backend.** The host has `bibtex` but not `biber`, so `biblatex` uses `backend=bibtex`.

12 AI-Use Note

Summarized from `AI_USE_LOG.md` in the code repository.

Tools used

Claude Code for coding, debugging, analysing the data and drafting this report. Claude Opus 5 did the initial harness and report, up to 2026-10-20. Claude Opus 5.5 did the 2026-10-21 rework and the analysis of the final runs. GCC gcc (GCC) 16.2.1 20260810 built the code. The AI wrote the code and the prose. The timed runs were executed by the author in a terminal on the machine in Table 1, using the scripts Claude Code wrote (`run.sh all`, then `SUSTAIN_GIB=60 run.sh device_state`, and later `rerun_clean.sh lean` after `sudo fstrim -v /`). Claude Code ran no timed benchmark; every number comes from those runs. Claude Code updated this report's text for the lean re-run.

What they helped generate

`blkbench.c` (the O_DIRECT `libaio` benchmark), the data-safety gate (`safety.sh`) and its tests (`validate.sh`), the experiment driver (`run.sh`) with its CSV reduction, the FIO job files and converter, and the first drafts of these sections.

What was independently modified, tested, or rejected

Concrete corrections:

1. *The wrong device was being tested.* The scratch directory defaulted to the in-repo `data/scratch/`, which lives on the 2 TB SATA *HDD*, so the first full dataset showed millisecond QD=1 latencies. The scratch path was made overridable (`ACS_SCRATCH`) and pointed at the NVMe SSD. The AI's claim that "the NVMe is read-only to us" was false, as a one-line write test showed. The device check in `env.txt` was itself buggy: it stripped trailing digits from `nvme0n1p3` and printed `rotational: ?`. It now resolves the partition to its parent disk.
2. An early version measured *submission* latency rather than *completion* latency; it now times each I/O to completion.
3. A convenience default left the page cache enabled and reported ∼2 million IOPS. That case became the required pitfall (Section 10), and `O_DIRECT` became the default.
4. The report claimed a `0xA5` write fill, but the timed write buffers were never initialised. They are now filled once with an incompressible xorshift pattern, and `make test` checks that it does not compress.
5. The report claimed `validate.sh` refused `/dev/nvme0n1` when it only tested `/dev/sda`; it now tests both. It also claimed a direct-vs-buffered assertion that did not exist; the assertion has been added.

6. An AI-drafted sentence said p99 “rises monotonically with write fraction”. The table contradicts it (pure write is below 50/50), and the sentence was corrected.
7. The first version of the sustained-write summary defined “burst” as the median of the first 10 s. On the real data, where the burst lasted only 3 s, that reported a misleading 66 MiB/s burst and a spurious cliff at 11 GiB. The definitions were fixed and the summary was regenerated from the unchanged raw log (`run.sh reprocess`).
8. The earlier report attributed the small (3.1 GiB) cache to the run order. The lean re-run on a trimmed, idle drive found 3.8 GiB, so that explanation was withdrawn (Section 9). The summary column `rand_steady_iops` is a byte-weighted phase average, and in the lean run a final 0.7 s at SLC speed doubles it. Checked against the per-second log, the text now quotes the per-second median as the steady value.
9. Claims carried over from the pilot data were re-checked against the final run and corrected where the data disagreed: the QD 1→4 scaling, the 26 % gain from QD 32 to 64 (15 %), the constant p99.9/p50 ratio, and the prediction that deep queues hide fragmentation.

Validation

`make test` checks the safety gate (both block devices, a symlink to one, `/etc/passwd`, `/`), the incompressible fill, physical orderings, the $L \leq \mathrm{QD}$ invariant, page-cache bypass and the FIO converter. FIO cross-checks `blkbench` (Section 3.1). Every figure traces to a committed raw/processed file (Section A).

13 Conclusion

The Micron 2210 NVMe SSD shows the queueing-driven throughput–latency tradeoff the project set out to measure. At QD=1, 4 KiB random reads cost 53.2 µs and writes only 16.0 µs, because the controller's write buffer hides the flash program.

Block size moves the bottleneck from per-request latency to bandwidth. With one request in flight, sequential reads stay near 1.4 GiB/s from 64 to 256 KiB and reach 1 798 MiB/s at 1 MiB. At QD=32 they top out at 2 144 MiB/s, 57 % of the 3756 MiB/s PCIe 3.0 ×4 link. The NAND back end, not the interface, is the ceiling. Writes saturate at ∼1.06 GiB/s of SLC ingest even at QD=1. Mixing reads with writes is worse than either pure workload.

The read knee is at QD 64 under an explicit 10 %-per-doubling rule. Beyond it, latency doubles with every doubling of QD for at most a few percent more IOPS. L/QD stays at 0.987–0.998. Little's Law holds by construction in a closed loop, so this confirms that the harness keeps the queue full and the saturation is the device's.

Device state matters as much as configuration. A lean re-run on a freshly trimmed, idle drive found the SLC cache much smaller than expected: a sequential fill hit the cliff after 3.8 GiB instead of the ∼20 GiB we had assumed, so the stop-at-cliff fill wrote only 4.3 GiB. That size is consistent with a dynamic cache that scales with the drive's free space, which we could not control or measure because only the scratch partition was trimmed. We report it as the cache available in this state, not as a fixed capacity. Past the cliff, sequential writes fall from 1 071 to 63 MiB/s (17×), and 4 KiB random writes run at about 15k IOPS with 36 ms per-second p99. A minute of idle restored at least 1 GiB of cache after this short fill, but nothing after the original run's 60 GiB one. On the rested drive, read tails widen with queue depth to p99.9/p50 = 5.7× at QD=128. Right after heavy writes they lost half their IOPS and reached 24×. Sequential reads of randomly written data run 6–9× slower whatever the queue depth. The write collapses seen inside the original sweeps disappeared when the same points were re-measured after a rest. The NVMe temperature peaked at 61.9 °C, far below the drive's 86.85 °C warning threshold, so we saw no thermal effects.

The sharpest lessons are methodological. Without `O_DIRECT`, the same workload reported page-cache fiction. The first dataset was taken on the HDD until the `rotational` flag gave it away. A "sequential" read can be six to nine times slower depending only on how the data was written. And on a QLC drive, *what ran just before* changes write throughput by an order of magnitude and read tails severalfold, and even a trimmed, idle drive offered an SLC cache far smaller than expected, plausibly because its size depends on how full the drive is. A storage measurement can be trusted only if it bypasses the cache, verifies its effective queue depth, controls the data's write history and the drive's recent workload, and confirms which device it is talking to.

References

[1] J. Axboe, *Fio: Flexible I/O tester*, Version 3.42 on the test host; used to cross-check the libaio benchmark (blkbench). [Online]. Available: `https://github.com/axboe/fio`

[2] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961. DOI: `10.1287/opre.9.3.383`

A Reproducibility

Code, scripts, raw data and this report live in one repository: `ACS-Code/` holds the code and data, and `ACS-Latex/` holds the reports. The exact commit is `64507e8a4d2e`.

```
gh repo clone SKempiak/ACS-Code ACS && cd ACS
git checkout 64507e8a4d2e
cd ACS-Code
make tools                                    # build/tools/summarize
make -C SSD_Performance_Profiling             # build/blkbench (O_DIRECT, libaio)
make -C SSD_Performance_Profiling test        # safety + sanity checks (~70 MiB written, deleted)
scripts/collect_env.sh --tex > ../ACS-Latex/common/system_info.tex
export ACS_SCRATCH="$HOME/acs_ssd_scratch"    # a directory on the SSD under test
SSD_Performance_Profiling/scripts/run.sh dryrun        # device, link, temperature, space; writes
    nothing
SSD_Performance_Profiling/scripts/run.sh all           # 2 GiB-file sweeps + fio_check (~23 min
    measured)
SUSTAIN_GIB=60 SSD_Performance_Profiling/scripts/run.sh device_state  # sustained + working set (
    needs 70 GiB free)
scripts/sync_data.sh SSD_Performance_Profiling
cd ../ACS-Latex && make ssd
```

Clean re-run. Several results of the original run were shaped by the run order: `all` followed directly by `device_state`, with no TRIM and no idle (Section 2). The full clean re-run below has not been run; the lean version after it has. `scripts/rerun_clean.sh` repeats the affected experiments in an order that avoids these confounds. It first checks free space (at least `SUSTAIN_GIB`+10 GiB), drive idleness (`/sys/block/nvme0n1/stat` over 10 s) and recent-TRIM evidence in the journal. It then runs:

1. the 60 GiB sustained fill on the fresh drive;
2. a 20 min idle, then the working set with spans forward and reverse;
3. a 10 min idle, then tail on a freshly preconditioned file;
4. zero-queue;
5. block-size, now with QD=32 random reads and a 15 s pause before each write point;

6. queue-depth;
7. `fio_check` with alternating engine order.

Steps 5–7 each follow a 5 min rest. The upper bound is ∼135 GiB written in ∼2 h; peak space in use is 60 GiB.

```
sudo fstrim -v /                # then leave the machine idle >= 30 min
export ACS_SCRATCH="$HOME/acs_ssd_scratch"
SSD_Performance_Profiling/scripts/rerun_clean.sh dryrun
SSD_Performance_Profiling/scripts/rerun_clean.sh
scripts/sync_data.sh SSD_Performance_Profiling
```

Lean re-run. `rerun_clean.sh lean` is a shorter version of the clean re-run: ∼35 GiB written expected, 55 GiB at most, in about 35 min to 50 min. Its steps are:

1. The sustained fill, first, that *stops at the cliff*: 10 seconds after throughput stays below 30 % of the burst median, capped at 40 GiB. With no cliff by then, the result is reported as a lower bound on the cache size. A short random phase, a 60 s idle and a recovery probe follow, and the large file is deleted.
2. A 10 min rest, then tail on a freshly preconditioned file.
3. The working set over 1, 2 and 4 GiB, forward and reverse.
4. QD=32 random reads, then the ≥256 KiB random writes again after a rest.
5. Write queue depths 4–32.
6. FIO write points with alternating engine order.

It also takes an NVMe SMART snapshot before and after, when `nvme smart-log` works without root. Partial re-measurements replace only the matching configurations of the 2026-10-21 run, and figure captions say which run each point comes from.

The lean re-run reported here has the timestamp `20261021-220614`. It was run on 2026-10-21 as follows. `sudo fstrim -v /` ran at 21:34:54, the machine was then left idle for at least 30 min, and the script's idle and TRIM checks passed. The run lasted from 22:06 to 22:32:17 EDT, with the governor at `performance`, turbo off, SMT off and ACPI GPE 0x42 masked. Its complete console output, including the free-space, TRIM-evidence and idle checks, the plan table and the per-minute temperature during the rests, is in `ACS-Code/SSD_Performance_Profiling/data/raw/rerun_clean_20261021-220614.log`. The SMART snapshots taken before and after the run (in `data/raw/smart/`) hold only the hwmon temperature, because `nvme smart-log` needs root. A second log, `rerun_clean_20261021-220544.log`, holds only the preflight lines of a start 30 s earlier that did not proceed to any measurement.

```
sudo fstrim -v /                # 21:34:54; then idle >= 30 min
export ACS_SCRATCH="$HOME/acs_ssd_scratch"
SSD_Performance_Profiling/scripts/rerun_clean.sh lean dryrun
SSD_Performance_Profiling/scripts/rerun_clean.sh lean   # 20261021-220614, 22:06-22:32 EDT
scripts/sync_data.sh SSD_Performance_Profiling
```

`ACS_SCRATCH` selects the device under test, and the `safety.sh` gate is identical wherever it points: a plain disposable file inside the scratch directory, never a device or mountpoint. Without `ACS_SCRATCH`, the in-repo `data/scratch/` filesystem is profiled instead (an HDD on the test host). Knobs (`REPS`, `RUNTIME`, `WRITE_CAP_MIB`, `SUSTAIN_GIB`, `ENGINE=fio`, ...) are documented at the top of `run.sh`. `run.sh reprocess` rebuilds every processed CSV from the newest raw run of each experiment.

The timed runs reported here were executed by the author in a terminal on the test machine, with the CPU governor set to `performance`, turbo off and SMT off. `run.sh all` started at 10:32

and `SUSTAIN_GIB=60 run.sh device_state` followed immediately at 10:54 on 2026-10-21; the lean re-run followed at 22:06 the same day.

Table 12: Raw data behind each figure (`ACS-Code/SSD_Performance_Profiling/data/raw/...`). Where two directories are listed, the lean (`_lean`) rows replace the matching configurations of the original run.

Figure / table	Raw run directory
Table 6	`zero_queue/20261021-103201`
Figures 1 and 2	`block_size/20261021-103201` + `block_size_lean/20261021-220614`
Figure 3, Table 7	`rw_mix/20261021-103201`
Figure 4, Table 8	`queue_depth/20261021-103201` + `queue_depth_lean/20261021-220614`
Figure 5	`tail/20261021-220614` (original run: `tail/20261021-103201`)
Table 11	`pitfall/20261021-103201`
Table 10	`layout/20261021-103201`
Tables 4 and 5	`fio_check/20261021-103201` + `fio_check_lean/20261021-220614` (FIO JSON under `fio/`)
Figure 6, Table 9	`sustained/20261021-220614` (original run: `sustained/20261021-105439`)
Figure 7	`working_set/20261021-220614` (original run: `working_set/20261021-105439`)
Table 3	`rerun_clean_20261021-220614.log`

Each raw directory holds `raw.csv` (one row per repetition or per second, with full configuration metadata and the NVMe temperature) and `env.txt`. `env.txt` contains the `collect_env.sh` snapshot plus the scratch filesystem's parent disk, its `rotational` flag, model, firmware, PCIe link, scheduler, temperature and thresholds, and the knob values. FIO runs also keep each JSON result under `fio/`. `tools/summarize` reduces the raw files to the processed CSVs plotted here, and `scripts/sync_data.sh` copies them into this report's `data/`. Tables and quoted numbers are read from those CSVs when the report is compiled, so every number traces to one of these files.